# ASSIGNMENT NO. 3 — COMPLEX PROBLEM-SOLVING CASE STUDY

# Online Shopping and Order Processing System

### Integrated Operating Systems Concepts

| | | | |
|---|---|---|---|
| **Name** | Tushar Tiple | **UID** | 24003145 |
| **Batch** | B2 | **Class** | 5th Sem, Section B |
| **Subject** | Operating Systems | **Project** | ShopOS |

## 1. Selected Problem Statement and Context

An e-commerce platform receives customer orders, processes payments, updates inventory, generates invoices and schedules deliveries. During a major sale, thousands of requests may arrive simultaneously. Tasks consume CPU time, memory and shared resources such as inventory services, payment workers and database connections. The system must process requests safely, prevent queue corruption and avoid unsafe resource allocation.

## 2. Objectives, Constraints and Assumptions

Objectives: (1) compare CPU scheduling using waiting and turnaround metrics; (2) demonstrate a bounded producer–consumer queue; (3) determine safe resource states with Banker's Algorithm; (4) compare First Fit, Best Fit and Worst Fit memory allocation; (5) connect all concepts to one order-processing flow.

**Assumptions:** times, resource counts and memory sizes are illustrative; priority 1 is highest; Banker's Algorithm uses declared maximum claims; memory allocation is contiguous. Payment, inventory and delivery are simulations only.

## 3. Integrated System Interface

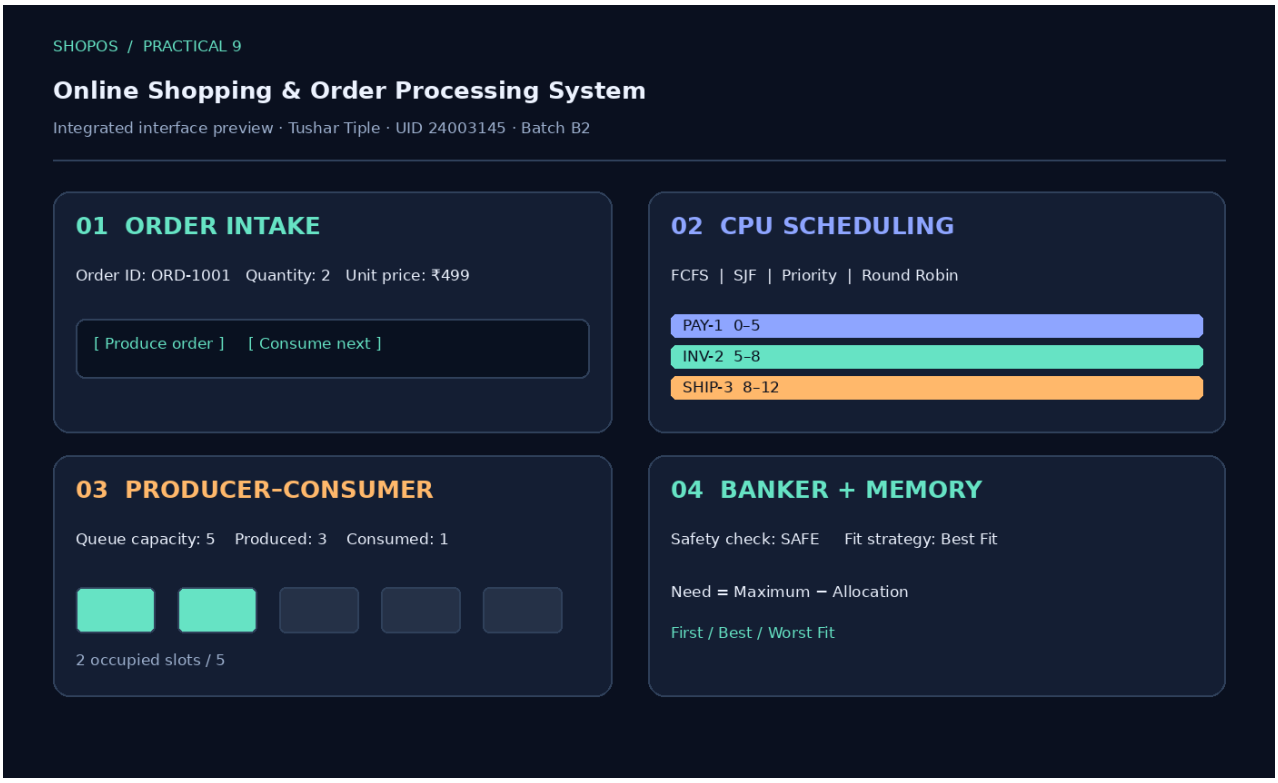



Figure 1. ShopOS interface preview showing the order intake and four OS concept panels.

The interface groups order intake, CPU scheduling, the synchronized queue, Banker's safety checking, memory allocation and order validation into one application.

## 4. CPU Scheduling

Order stages such as payment validation, inventory checking and delivery scheduling are represented as processes. FCFS follows arrival order; SJF selects the shortest ready task; Priority selects the lowest priority number; Round Robin uses a time quantum.

**Formulas:** Turnaround = Completion – Arrival; Waiting = Turnaround – Burst; Response = First Start – Arrival.

Illustrative tasks: PAY-1 (arrival 0, burst 5, priority 2), INV-2 (arrival 1, burst 3, priority 1), SHIP-3 (arrival 2, burst 4, priority 3).




Figure 2. FCFS execution segments and per-process timing metrics.

## 5. Process Synchronization — Producer–Consumer

Incoming orders are producers; processing workers are consumers. A bounded queue limits backlog, and a lock protects shared queue/counter state. A full queue reports that a producer must wait; an empty queue reports that a consumer must wait. Invariant: produced – consumed = items currently in queue.

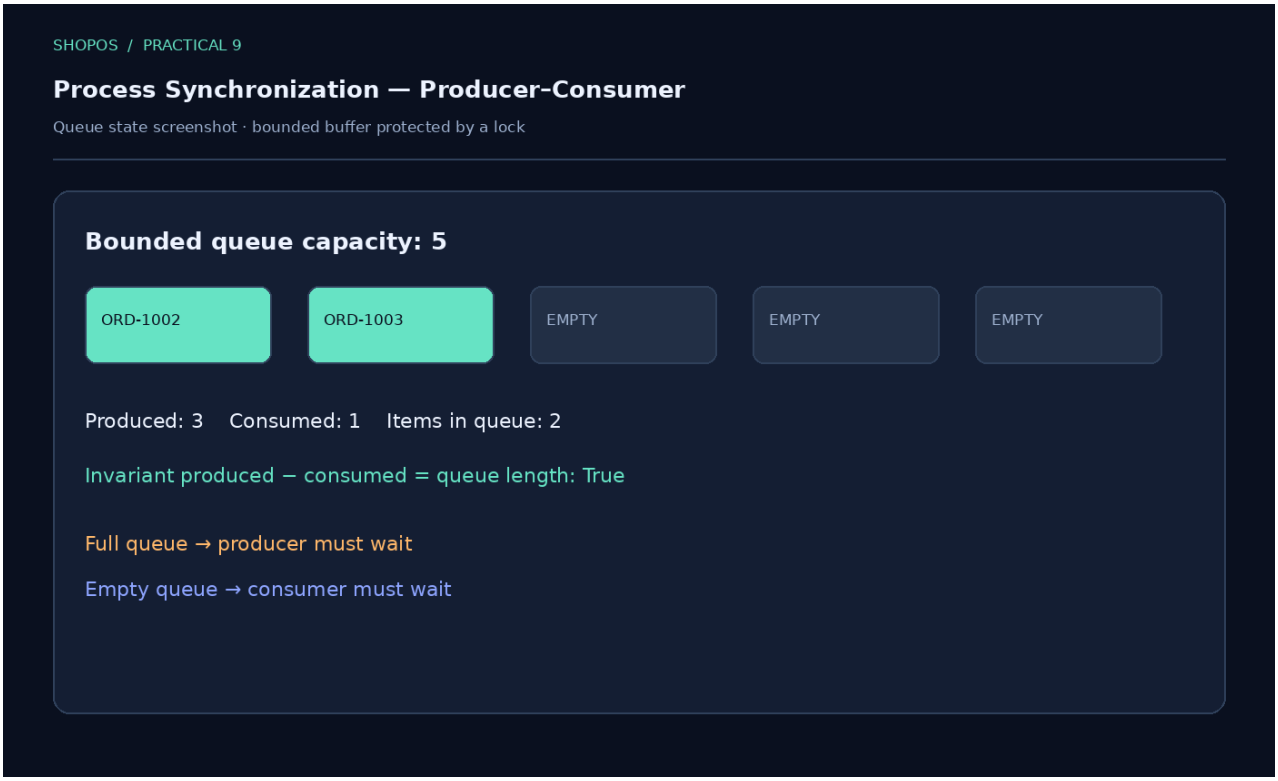



Figure 3. Bounded queue contents, counters and synchronization invariant.

## 6. Deadlock Avoidance — Banker’s Algorithm

Resource types represent payment workers, inventory connections and database connections. Need = Maximum – Allocation. Starting with Work = Available, the algorithm finds an unfinished process whose Need ≤ Work, simulates its completion and returns its allocation to Work. If every process can finish, the state is safe and the resulting order is a safe sequence.

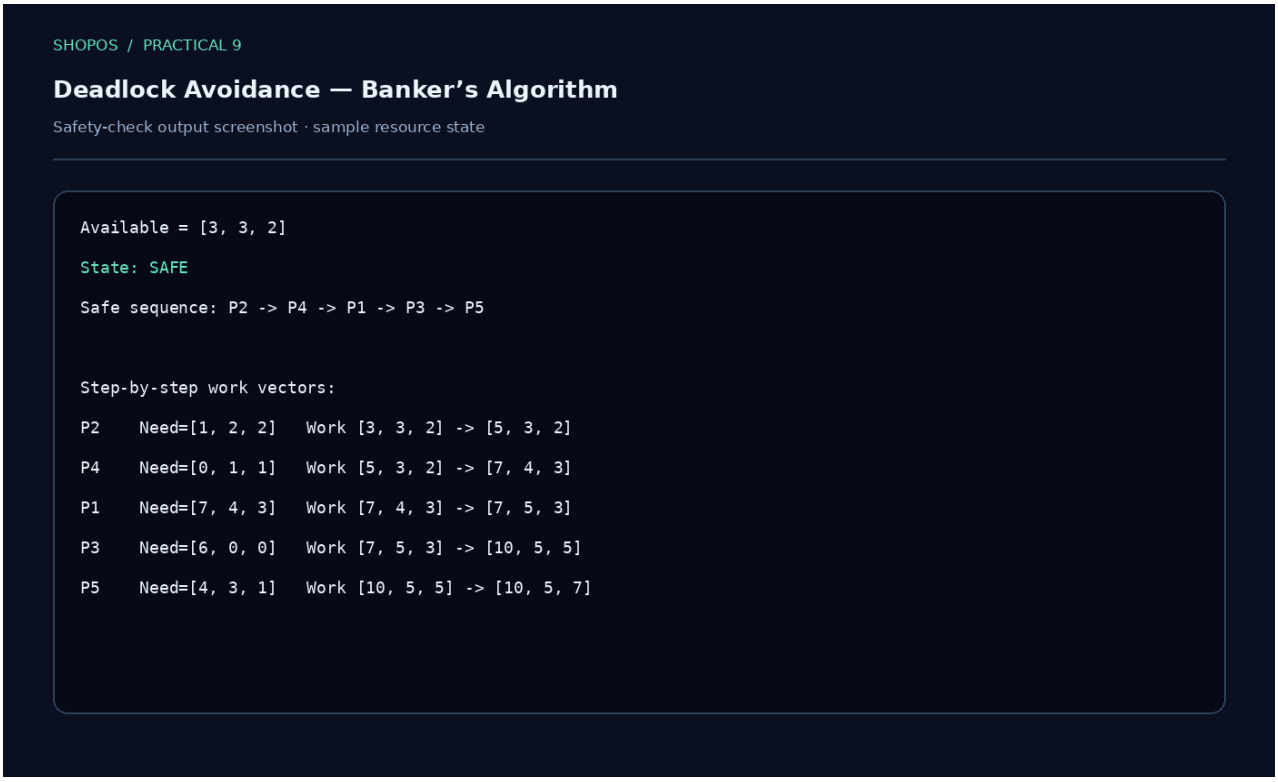



Figure 4. Safe-state result, safe sequence and Work-vector progression.

## 7. Memory Management

Order payloads, invoice data and delivery metadata request memory. First Fit selects the first adequate block; Best Fit selects the smallest adequate block; Worst Fit selects the largest adequate block. The report shows allocation decisions and remaining space.




Figure 5. Best Fit allocation output and remaining free blocks.

## 8. Order Processing and Integration

The order validator checks positive quantity and non-negative unit price, calculates totals and assigns simulated invoice identifiers. No payment is charged, stock is changed or courier is booked.




Figure 6. Order validation, invoice totals and explicitly simulated service statuses.

## 9. Test Plan

| Module | Test | Expected result |
|---|---|---|
| CPU scheduling | Different arrivals/bursts; RR quantum | Execution segments and timing metrics |
| Producer–Consumer | Queue full/empty and normal operations | Blocked status and counter invariant |
| Banker's Algorithm | Safe, unsafe and invalid claims | Safe sequence or unsafe state |
| Memory | Fit/no-fit across strategies | Allocation and remainder per block |
| Order validation | Valid and invalid order fields | Calculated total or rejection |

## 10. Limitations and Conclusion

ShopOS is an educational simulator, not a production commerce platform. It has no authentication, persistent database, real payment integration, stock reservation or courier API. The queue API reports blocked conditions rather than suspending HTTP requests, and Banker's Algorithm is a safety check rather than a full atomic resource manager.

The project integrates all four required concepts: scheduling analyses task latency, synchronization protects shared queue state, Banker's Algorithm checks safe resource progress, and memory strategies compare allocation and fragmentation.

**Run:** extract the repository ZIP, install dependencies with `pip install -r requirements.txt`, run `python app.py`, then open `http://127.0.0.1:5000`.